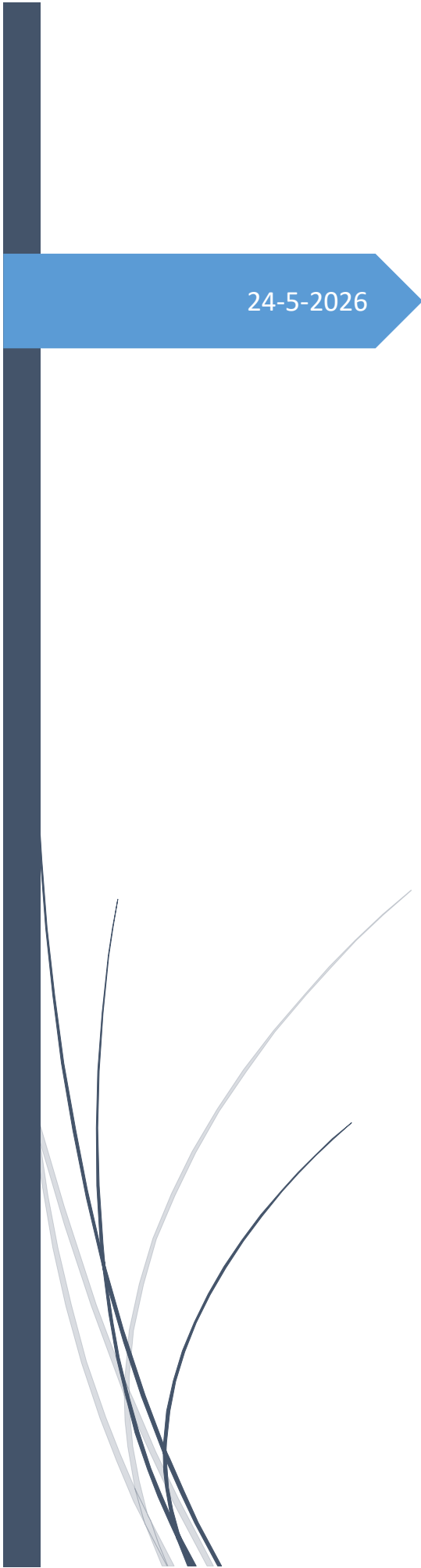




24-5-2026

## Ejercicio 2:

Propuesta de arquitectura de  
datos en AWS

Diego Atzin Pineda Cota

## Contenido

|                                                               |    |
|---------------------------------------------------------------|----|
| <b>Ejercicio 2: Propuesta de arquitectura de datos en AWS</b> | 4  |
| <b>1. Introducción</b>                                        | 4  |
| 2. Arquitectura propuesta                                     | 5  |
| I. Propuesta de arquitectura                                  | 6  |
| A. Subconjunto de datos a extraer de cada fuente              | 6  |
| Fuente F1: CRM propietario                                    | 6  |
| Fuente F2: SQL Server                                         | 6  |
| Fuente F3: PostgreSQL                                         | 7  |
| B. Retos de extracción por fuente y herramientas utilizadas   | 8  |
| F1: CRM propietario                                           | 8  |
| Retos                                                         | 8  |
| Herramientas propuestas                                       | 8  |
| F2: SQL Server                                                | 9  |
| Retos                                                         | 9  |
| Herramientas propuestas                                       | 9  |
| F3: PostgreSQL                                                | 9  |
| Retos                                                         | 9  |
| Herramientas propuestas                                       | 9  |
| C. Retos por independencia en el modelo de datos y solución   | 10 |
| D. Mitigación del impacto sobre las fuentes originales        | 11 |
| 1. Extracción incremental                                     | 11 |
| 2. CDC como evolución                                         | 11 |
| 3. Consultas paginadas                                        | 11 |
| 4. Índices en campos de extracción                            | 11 |
| 6. Control de concurrencia                                    | 12 |
| E. Etapas del proceso de transformación                       | 12 |
| 1. Landing                                                    | 12 |
| Uso                                                           | 12 |
| 2. Bronze                                                     | 12 |
| Uso                                                           | 13 |
| 3. Silver                                                     | 13 |
| Uso                                                           | 13 |

|                                                               |    |
|---------------------------------------------------------------|----|
| 4. Gold .....                                                 | 13 |
| F. Herramientas para las etapas de transformación .....       | 15 |
| AWS Glue .....                                                | 15 |
| PySpark .....                                                 | 15 |
| dbt como complemento opcional .....                           | 15 |
| G. Storage para cada propósito .....                          | 16 |
| 1. Amazon S3 como Data Lake principal .....                   | 16 |
| Justificación .....                                           | 16 |
| 2. Amazon Athena para consultas SQL .....                     | 16 |
| Justificación .....                                           | 16 |
| 3. S3 Gold Data Science para ciencia de datos .....           | 17 |
| Uso .....                                                     | 17 |
| 4. Motor de grafos .....                                      | 17 |
| Justificación .....                                           | 17 |
| H. Orquestación del pipeline .....                            | 18 |
| Justificación .....                                           | 18 |
| Alternativas de despliegue de Airflow .....                   | 18 |
| Uso de Lambda y EventBridge .....                             | 19 |
| II. Seguridad .....                                           | 20 |
| A. Seguridad end-to-end del flujo de datos .....              | 20 |
| 1. Seguridad de red .....                                     | 20 |
| 2. Seguridad de credenciales .....                            | 20 |
| 3. Cifrado .....                                              | 21 |
| 4. Control de acceso .....                                    | 21 |
| 5. Protección de datos sensibles .....                        | 22 |
| 6. Monitoreo y alertas .....                                  | 22 |
| III. Gobernanza de datos .....                                | 23 |
| A. Control de metadata, cambios y procesos del pipeline ..... | 23 |
| 1. Metadata técnica .....                                     | 23 |
| 2. Control de cambios de esquema .....                        | 24 |
| 3. Control de procesos del pipeline .....                     | 25 |
| 4. Linaje de datos .....                                      | 25 |
| 4. Componentes complementarios .....                          | 26 |

|                            |    |
|----------------------------|----|
| Kafka / Amazon MSK.....    | 26 |
| Grafana .....              | 26 |
| Lambda y EventBridge ..... | 27 |
| 5. Conclusión.....         | 27 |

# Ejercicio 2: Propuesta de arquitectura de datos en AWS

## 1. Introducción

Se propone una arquitectura de datos en AWS para consolidar información proveniente de tres fuentes operativas transaccionales que funcionan 24/7:

- **F1:** CRM propietario con información demográfica y de contacto de clientes.
- **F2:** RDBMS SQL Server con transacciones de clientes sobre una parte de los productos.
- **F3:** RDBMS PostgreSQL con transacciones de clientes sobre el resto de los productos.

El objetivo principal de la arquitectura es habilitar al área operativa para consultar información consolidada mediante SQL. Como objetivo secundario, la arquitectura debe permitir al equipo de ciencia de datos aplicar algoritmos de detección de patrones, como clustering o búsquedas en grafos.

Para resolver este escenario, se propone una arquitectura tipo **Data Lakehouse en AWS**, utilizando Amazon S3 como almacenamiento central, AWS Glue como motor de transformación, AWS Glue Data Catalog como catálogo de metadata, Amazon Athena como motor SQL y Apache Airflow, preferentemente mediante Amazon MWAA, como herramienta de orquestación.

## 2. Arquitectura propuesta

La arquitectura se compone de las siguientes capas y servicios:

| Componente                    | Uso dentro de la arquitectura                                  |
|-------------------------------|----------------------------------------------------------------|
| Amazon S3                     | Almacenamiento central del Data Lake                           |
| AWS Glue                      | Transformaciones ETL con PySpark                               |
| AWS Glue Data Catalog         | Catálogo técnico de tablas y metadata                          |
| Amazon Athena                 | Consultas SQL sobre datos en S3                                |
| Apache Airflow / Amazon MWAA  | Orquestación del pipeline                                      |
| AWS Lambda                    | Validaciones ligeras, triggers y automatizaciones event-driven |
| Amazon EventBridge            | Programación de ejecuciones y disparo de eventos               |
| Amazon CloudWatch             | Logs, métricas y alertas                                       |
| Grafana                       | Dashboards de monitoreo y observabilidad                       |
| Amazon SageMaker Notebooks    | Ciencia de datos y modelos de clustering                       |
| Amazon Neptune / Neo4j        | Análisis y búsquedas en grafos                                 |
| Kafka / Amazon MSK / Debezium | Evolución opcional para CDC o near real-time                   |

La arquitectura principal trabaja con un proceso **batch incremental diario**, debido a que el requerimiento indica que diariamente existen datos nuevos. Sin embargo, se contempla una evolución mediante CDC y Kafka para reducir el impacto sobre las fuentes y habilitar procesamiento near real-time si el negocio lo requiere.

# I. Propuesta de arquitectura

## A. Subconjunto de datos a extraer de cada fuente

No se recomienda extraer toda la información de las fuentes, sino únicamente los campos necesarios para cumplir los objetivos operativos y analíticos.

### Fuente F1: CRM propietario

De F1 se extraería la información relacionada con clientes.

| Tipo de dato               | Ejemplos de campos                             |
|----------------------------|------------------------------------------------|
| Identificación del cliente | customer_id, external_customer_id, document_id |
| Datos demográficos         | edad, género, ubicación, segmento              |
| Datos de contacto          | email, teléfono, dirección                     |
| Estado del cliente         | activo, inactivo, fecha_alta, fecha_baja       |
| Segmentación               | tipo_cliente, categoría, score interno         |
| Control de cambios         | created_at, updated_at                         |

#### Justificación:

F1 funcionaría como la fuente principal de información del cliente. Estos datos permiten construir una vista consolidada del cliente, enriquecer las transacciones de F2 y F3, y generar variables relevantes para ciencia de datos, como segmentación, perfilamiento y análisis de comportamiento.

### Fuente F2: SQL Server

De F2 se extraería información transaccional relacionada con los productos administrados por esta fuente.

| Tipo de dato                  | Ejemplos de campos              |
|-------------------------------|---------------------------------|
| Identificación de transacción | transaction_id                  |
| Identificación del cliente    | customer_id o clave equivalente |
| Producto                      | product_id, product_type        |
| Monto                         | amount, currency                |
| Fecha de transacción          | transaction_date                |
| Estado                        | status, payment_status          |
| Control de cambios            | created_at, updated_at          |

#### Justificación:

F2 contiene una parte del comportamiento transaccional del cliente. Esta información permite construir métricas como frecuencia de compra, monto acumulado, productos adquiridos, comportamiento por canal y evolución transaccional

## Fuente F3: PostgreSQL

De F3 se extraería un subconjunto similar al de F2.

| Tipo de dato                  | Ejemplos de campos              |
|-------------------------------|---------------------------------|
| Identificación de transacción | transaction_id                  |
| Identificación del cliente    | customer_id o clave equivalente |
| Producto                      | product_id, product_type        |
| Monto                         | amount, currency                |
| Fecha de transacción          | transaction_date                |
| Estado                        | status                          |
| Control de cambios            | created_at, updated_at          |

#### Justificación:

F3 complementa la información transaccional de F2, ya que contiene operaciones sobre el resto de los productos de la compañía. Ambas fuentes deben integrarse en un modelo común de transacciones para lograr una vista completa del comportamiento del cliente.



## B. Retos de extracción por fuente y herramientas utilizadas

### F1: CRM propietario

#### Retos

F1 puede presentar retos particulares porque es un CRM propietario. Esto puede implicar que no exista acceso directo a la base de datos, que la extracción dependa de una API, que existan límites de consumo, paginación o formatos específicos como JSON, XML o CSV.

También puede existir dificultad para identificar cambios incrementales si la fuente no cuenta con campos como `updated_at` o mecanismos de auditoría.

#### Herramientas propuestas

- API REST o conector propietario.
- Python para consumo de API.
- Airflow / Amazon MWAA para orquestar la extracción.
- AWS Secrets Manager para gestión segura de credenciales.
- Amazon S3 para almacenar la información cruda en Landing.
- AWS Lambda como apoyo para validaciones ligeras o disparadores.

## F2: SQL Server

### Retos

F2 es una base transaccional que funciona 24/7. Por lo tanto, una extracción mal diseñada podría afectar el rendimiento operativo, generar bloqueos o aumentar la carga en horarios críticos.

También pueden existir retos de volumen, tipos de datos, índices insuficientes y necesidad de extraer solo registros nuevos o modificados.

### Herramientas propuestas

- JDBC para conexión.
- Airflow / Amazon MWAA para orquestación.
- AWS Glue para extracción y transformación.
- SQL Server CDC si está disponible.
- Réplica de lectura si la arquitectura operativa lo permite.
- Debezium + Kafka / Amazon MSK como evolución para CDC.
- Amazon S3 como destino inicial en Landing.

## F3: PostgreSQL

### Retos

F3 también es una fuente transaccional 24/7. Puede tener diferencias de esquema frente a SQL Server, distintos tipos de datos, formatos de fecha diferentes y reglas de negocio propias.

Además, se debe cuidar que las consultas de extracción no afecten la operación diaria.

### Herramientas propuestas

- JDBC.
- Airflow / Amazon MWAA.
- AWS Glue.
- PostgreSQL logical replication como alternativa CDC.
- Debezium + Kafka / Amazon MSK para captura de cambios near real-time.
- Amazon S3 para almacenamiento en Landing.

## C. Retos por independencia en el modelo de datos y solución

Como F1, F2 y F3 fueron diseñadas de forma independiente, pueden existir diferencias en nombres de columnas, tipos de datos, identificadores, catálogos, reglas de negocio y niveles de calidad.

| Reto                                  | Solución propuesta                      |
|---------------------------------------|-----------------------------------------|
| Diferentes identificadores de cliente | Crear una llave maestra de cliente      |
| Nombres de columnas distintos         | Definir un modelo canónico              |
| Tipos de datos diferentes             | Normalizar tipos en la capa Silver      |
| Diferentes catálogos de productos     | Crear dimensión homologada de productos |
| Fechas en diferente formato           | Estandarizar fechas y zona horaria      |
| Clientes duplicados                   | Aplicar reglas de deduplicación         |
| Estados transaccionales diferentes    | Homologar estados en un catálogo común  |
| Reglas de negocio distintas           | Documentar reglas y versionarlas        |

La solución principal consiste en crear un **modelo canónico de datos**. Este modelo permite que las transacciones de F2 y F3 se transformen hacia una estructura común, mientras F1 se utiliza como dimensión principal de clientes.

### Ejemplo de modelo canónico de transacciones:

| Campo canónico      | Descripción                           |
|---------------------|---------------------------------------|
| customer_key        | Identificador homologado del cliente  |
| transaction_key     | Identificador único de transacción    |
| product_key         | Identificador homologado del producto |
| source_system       | Fuente de origen: F2 o F3             |
| transaction_amount  | Monto de la transacción               |
| currency            | Moneda                                |
| transaction_date    | Fecha de operación                    |
| transaction_status  | Estado homologado                     |
| ingestion_timestamp | Fecha de ingesta                      |
| batch_id            | Identificador de ejecución            |

## D. Mitigación del impacto sobre las fuentes originales

Además de ejecutar el proceso batch en la hora de menor uso, se aplicarían varias estrategias para disminuir el impacto sobre las fuentes transaccionales.

### 1. Extracción incremental

Se extraerían únicamente registros nuevos o modificados usando campos como:

- `created_at`
- `updated_at`
- `transaction_date`
- `id_incremental`

Esto evita extraer tablas completas diariamente y reduce la carga sobre las fuentes.

### 2. CDC como evolución

Para escenarios donde se requiera menor latencia o menor impacto sobre las fuentes, se podría implementar CDC.

Ejemplo

Event > Kafka/ Airflow > aws s3 Landing/Bronze

Kafka no reemplaza a Airflow. Kafka transporta eventos o cambios de datos, mientras que Airflow coordina las etapas del pipeline.

### 3. Consultas paginadas

Para evitar consultas grandes, las extracciones se dividirían por:

- Rangos de fechas.
- Rangos de ID.
- Tamaño de lote.
- Ventanas de extracción.

### 4. Índices en campos de extracción

Los campos usados para extracción incremental deberían estar indexados, esto permite que las consultas incrementales sean más eficientes.

## **6. Control de concurrencia**

El pipeline debe limitar el número de conexiones simultáneas hacia las fuentes. Esto evita saturar el CRM propietario.

# **E. Etapas del proceso de transformación**

Se propone dividir el procesamiento en cuatro capas: Landing, Bronze, Silver y Gold.

## **1. Landing**

La capa Landing almacena los datos tal como llegan desde las fuentes.

### **Uso**

- Mantener copia original.
- Permitir auditoría.
- Permitir reprocesos.
- Separar extracción de transformación.
- Guardar evidencia de lo recibido desde cada fuente.

Ejemplo de rutas:

- s3://data-lake/landing/f1\_crm/ingestion\_date=2026-05-24/
- s3://data-lake/landing/f2\_sqlserver/ingestion\_date=2026-05-24/
- s3://data-lake/landing/f3\_postgresql/ingestion\_date=2026-05-24/

## **2. Bronze**

La capa Bronze contiene datos crudos estructurados.

#### Uso

- Convertir datos a formato Parquet.
- Agregar columnas técnicas.
- Validar estructura mínima.
- Mantener datos cercanos al origen.
- Facilitar reprocesamiento eficiente.

Columnas técnicas recomendadas:

| Columna             | Uso                        |
|---------------------|----------------------------|
| source_system       | Identifica F1, F2 o F3     |
| ingestion_timestamp | Fecha y hora de ingesta    |
| batch_id            | Identificador de ejecución |
| record_hash         | Control de cambios         |
| raw_file_path       | Ruta del archivo origen    |

## 3. Silver

La capa Silver contiene datos limpios, validados y homologados.

#### Uso

- Estandarizar nombres de columnas.
- Convertir tipos de datos.
- Homologar catálogos.
- Resolver claves de cliente.
- Deduplicar registros.
- Unificar transacciones de F2 y F3.
- Aplicar reglas de calidad.

## 4. Gold

La capa Gold contiene datos listos para consumo.

Se propone dividirla en dos salidas principales.

- **Gold operacional**
  - Orientada a usuarios operativos y consultas SQL.
  - Esta capa sería consultada principalmente desde Amazon Athena.
- **Gold ciencia de datos**
  - Orientada a clustering, análisis de patrones y grafos.
  - Esta capa puede ser consumida desde notebooks, SageMaker, Spark ML o motores de grafos.

## F. Herramientas para las etapas de transformación

Para las etapas de transformación se utilizaría principalmente **AWS Glue con PySpark**.

### AWS Glue

AWS Glue se usaría para:

- Leer datos desde S3 Landing.
- Convertir datos a Parquet.
- Construir capas Bronze, Silver y Gold.
- Ejecutar transformaciones distribuidas.
- Integrarse con AWS Glue Data Catalog.
- Registrar tablas para consumo desde Athena.

### PySpark

PySpark permite realizar transformaciones como:

- Limpieza de datos.
- Joins entre fuentes.
- Deduplicación.
- Agregaciones.
- Normalización de columnas.
- Cálculo de métricas.
- Construcción de features.
- Generación de relaciones para grafos.

### dbt como complemento opcional

Para la capa Gold se puede considerar dbt si se desea modelar transformaciones SQL de forma versionada y documentada.

dbt puede ayudar a:

- Versionar modelos SQL.
- Documentar transformaciones.
- Crear pruebas de calidad.
- Facilitar mantenimiento de modelos analíticos.



# **G. Storage para cada propósito**

## **1. Amazon S3 como Data Lake principal**

Se utilizaría Amazon S3 para almacenar las capas Landing, Bronze, Silver y Gold.

### **Justificación**

Amazon S3 es adecuado porque:

- Es escalable.
- Es económico.
- Permite almacenar histórico.
- Se integra con Glue, Athena, SageMaker y otros servicios de AWS.
- Permite particionamiento.
- Permite usar formatos columnares como Parquet.

## **2. Amazon Athena para consultas SQL**

Para usuarios operativos se utilizaría Amazon Athena consultando tablas registradas en AWS Glue Data Catalog.

### **Justificación**

Athena permite:

- Ejecutar consultas SQL directamente sobre S3.
- Evitar administrar servidores.
- Consultar datos en formato Parquet.
- Aprovechar particionamiento.
- Integrarse con Glue Data Catalog.
- Habilitar consumo analítico sin mover los datos a otra base.

### 3. S3 Gold Data Science para ciencia de datos

Para ciencia de datos se almacenarán datasets preparados en S3, en formato Parquet.

#### Uso

- Clustering.
- Segmentación de clientes.
- Detección de patrones.
- Construcción de features.
- Análisis de comportamiento transaccional.

Estos datasets podrían consumirse desde:

- Amazon SageMaker.
- Notebooks.
- AWS Glue.
- EMR.
- Spark ML.

### 4. Motor de grafos

Para búsquedas en grafos, se generarían relaciones desde la capa Gold Data Science.

Estas relaciones podrían cargarse en:

- Amazon Neptune.
- Neo4j.

#### Justificación

Un motor de grafos permite analizar relaciones complejas, como comunidades de clientes, productos relacionados, rutas de comportamiento, afinidad de productos o patrones de conexión entre entidades.

# H. Orquestación del pipeline

El pipeline se orquestaría con **Apache Airflow**.

## Justificación

Airflow permite:

- Programar ejecuciones diarias.
- Definir dependencias entre tareas.
- Reintentar tareas fallidas.
- Registrar logs.
- Ejecutar procesos por fecha.
- Controlar backfills.
- Integrarse con AWS Glue, S3, Athena y bases externas.
- Monitorear el estado de cada etapa.

## Alternativas de despliegue de Airflow

Apache Airflow puede ejecutarse de diferentes formas en AWS:

| Opción                  | Uso recomendado                                                  |
|-------------------------|------------------------------------------------------------------|
| Amazon MWAA             | Producción administrada                                          |
| EC2 + Docker<br>Compose | Prototipo o prueba técnica                                       |
| ECS / Fargate           | Airflow en contenedores sin administrar servidores               |
| EKS                     | Airflow sobre Kubernetes en organizaciones con Kubernetes maduro |

Para una arquitectura productiva se recomienda Amazon MWAA o ECS/Fargate. Para un entorno de prueba, una instancia EC2 con Docker puede ser suficiente.

# Uso de Lambda y EventBridge

Adicionalmente, la arquitectura puede incorporar servicios serverless como AWS Lambda y Amazon EventBridge.

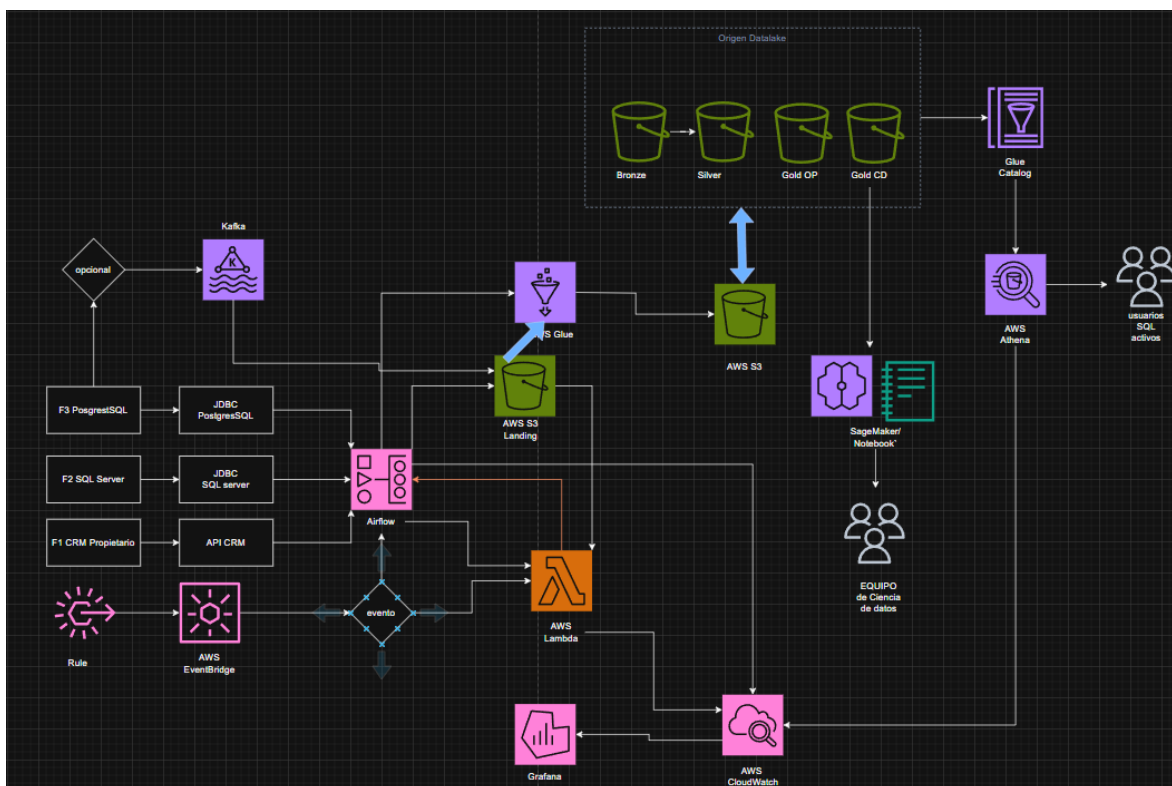
EventBridge puede utilizarse para:

- Programar ejecuciones diarias.
- Disparar eventos operativos.
- Activar una Lambda.
- Activar un flujo de validación.

Lambda puede utilizarse para:

- Validar llegada de archivos a S3.
- Registrar metadata de ingesta.
- Ejecutar validaciones ligeras.
- Disparar un DAG o un Glue Job.
- Enviar notificaciones.
- Reaccionar a eventos de S3.

Sin embargo, Lambda y EventBridge no reemplazan a Airflow cuando el flujo tiene múltiples dependencias. Airflow se mantiene como orquestador principal del pipeline.



## **II. Seguridad**

### **A. Seguridad end-to-end del flujo de datos**

Para disminuir riesgos de fuga de información o intrusiones no deseadas, se aplicarían controles en red, identidad, almacenamiento, cifrado, auditoría y gobierno de accesos.

#### **1. Seguridad de red**

Las fuentes operativas no deben exponerse públicamente.

Medidas propuestas:

- Ejecutar procesos dentro de una VPC privada.
- Usar subredes privadas para Airflow/MWAA y Glue.
- Acceder a SQL Server y PostgreSQL mediante red privada.
- Usar VPN, Direct Connect, VPC Peering o PrivateLink si aplica.
- Restringir conexiones mediante Security Groups.
- Evitar exposición pública de bases de datos.
- Permitir únicamente conexiones desde roles, subredes o IPs autorizadas.

#### **2. Seguridad de credenciales**

Las credenciales no deben almacenarse en código, archivos planos o repositorios.

Se utilizaría:

- AWS Secrets Manager.
- IAM Roles.
- Rotación de credenciales.
- Políticas de mínimo privilegio.
- Separación de credenciales por fuente.
- Acceso controlado desde Airflow y Glue.

### Ejemplo:

| Componente          | Acceso mínimo                                      |
|---------------------|----------------------------------------------------|
| Airflow / MWAA      | Ejecutar Glue Jobs y leer secrets necesarios       |
| Glue                | Leer S3 Landing y escribir Bronze/Silver/Gold      |
| Athena              | Leer tablas autorizadas                            |
| Usuarios operativos | Consultar solo Gold operacional                    |
| Ciencia de datos    | Consultar Gold Data Science y datasets autorizados |

## 3. Cifrado

Se aplicaría cifrado en tránsito y en reposo.

| Capa              | Medida                    |
|-------------------|---------------------------|
| API/JDBC          | TLS/SSL                   |
| Amazon S3         | SSE-KMS                   |
| Glue Data Catalog | Cifrado con KMS           |
| Secrets           | Secrets Manager con KMS   |
| Logs              | CloudWatch cifrado        |
| Athena results    | Resultados en S3 cifrados |
| Backups           | Cifrado obligatorio       |

## 4. Control de acceso

El acceso se controlaría mediante IAM y Lake Formation.

Se usaría:

- IAM Roles.
- AWS Lake Formation.
- Políticas por base de datos.
- Políticas por tabla.
- Políticas por columna.
- Separación de permisos por rol.
- Auditoría de accesos.

Ejemplo:

| Rol               | Acceso                                    |
|-------------------|-------------------------------------------|
| Data Engineer     | Landing, Bronze, Silver y Gold            |
| Usuario operativo | Solo Gold operacional                     |
| Data Scientist    | Gold Data Science y algunas tablas Silver |
| Auditor           | Metadata, logs y linaje                   |
| Administrador     | Gestión controlada del entorno            |

## 5. Protección de datos sensibles

Como F1 contiene datos personales y de contacto, se deben aplicar medidas especiales.

Medidas propuestas:

- Enmascaramiento de email y teléfono para usuarios no autorizados.
- Hashing o tokenización de identificadores sensibles.
- Separación de PII en tablas restringidas.
- Acceso por columna mediante Lake Formation.
- Auditoría de consultas sobre datos sensibles.
- Clasificación de datasets según nivel de sensibilidad.

## 6. Monitoreo y alertas

Se utilizaría CloudWatch y Grafana para monitorear:

- Estado de DAGs.
- Duración de ejecuciones.
- Jobs de Glue fallidos.
- Errores de extracción.
- Volumen de registros procesados.
- Registros rechazados.
- Consultas Athena.
- Métricas de infraestructura.
- Métricas de Kafka si se usa CDC.

Grafana no reemplaza a Airflow. Grafana permite visualizar métricas y detectar comportamientos anómalos, mientras que Airflow coordina la ejecución del pipeline.

# III. Gobernanza de datos

## A. Control de metadata, cambios y procesos del pipeline

La gobernanza se apoyaría en los siguientes componentes:

| Componente                        | Uso                                      |
|-----------------------------------|------------------------------------------|
| AWS Glue Data Catalog             | Catálogo técnico de tablas               |
| AWS Lake Formation                | Gobierno y permisos sobre datos          |
| Tablas de auditoría               | Control de ejecuciones y cambios         |
| CloudWatch                        | Logs y monitoreo                         |
| dbt Docs / OpenMetadata / DataHub | Documentación y linaje avanzado opcional |

### 1. Metadata técnica

Glue Data Catalog almacenaría metadata de:

- Bases de datos.
- Tablas.
- Columnas.
- Tipos de datos.
- Particiones.
- Ubicaciones en S3.
- Formato de archivo.
- Esquemas.



## 2. Control de cambios de esquema

Se implementaría validación de esquemas en cada ejecución del pipeline.

Se controlaría:

- Columnas nuevas.
- Columnas eliminadas.
- Cambios de tipo de dato.
- Cambios de nulabilidad.
- Cambios de formato.
- Cambios de catálogo.

Ejemplo de tabla de control de esquema:

| Campo          | Descripción                  |
|----------------|------------------------------|
| dataset_name   | Nombre del dataset           |
| source_system  | F1, F2 o F3                  |
| schema_version | Versión del esquema          |
| column_name    | Nombre de columna            |
| data_type      | Tipo de dato                 |
| is_nullable    | Permite nulos                |
| detected_at    | Fecha de detección           |
| change_type    | Nuevo, eliminado, modificado |

Esta tabla podría almacenarse en S3 y consultarse desde Athena, o en una base PostgreSQL de metadata.

### 3. Control de procesos del pipeline

Cada ejecución del pipeline debe registrar información operativa.

Ejemplo de tabla de auditoría:

| Campo            | Descripción              |
|------------------|--------------------------|
| pipeline_name    | Nombre del pipeline      |
| dag_id           | DAG de Airflow           |
| run_id           | ID de ejecución          |
| batch_id         | ID del lote              |
| source_system    | Fuente procesada         |
| status           | Success, failed, running |
| start_time       | Hora de inicio           |
| end_time         | Hora de fin              |
| records_read     | Registros leídos         |
| records_written  | Registros escritos       |
| rejected_records | Registros rechazados     |
| error_message    | Error si aplica          |
| output_path      | Ruta de salida en S3     |

Esta metadata puede almacenarse en:

- Amazon S3 como tabla de auditoría.
- Amazon RDS PostgreSQL.
- Metadata DB de Airflow.
- Glue Data Catalog para consulta vía Athena.

### 4. Linaje de datos

El linaje permitiría responder preguntas como:

- ¿De qué fuente viene este dato?
- ¿Qué transformación se aplicó?
- ¿Qué pipeline generó esta tabla?
- ¿Qué ejecución produjo este resultado?
- ¿Qué columnas fueron modificadas?

Para esto se usarían columnas técnicas en las capas del Data Lake

| Columna             | Uso                       |
|---------------------|---------------------------|
| source_system       | Fuente origen             |
| ingestion_timestamp | Fecha de ingesta          |
| batch_id            | Lote de ejecución         |
| record_hash         | Validación de cambios     |
| pipeline_version    | Versión del pipeline      |
| raw_file_path       | Ruta del archivo original |

## 4. Componentes complementarios

### Kafka / Amazon MSK

Kafka se considera un componente opcional o evolutivo. Su uso principal sería capturar cambios desde las fuentes transaccionales mediante CDC, reduciendo el impacto de consultas batch sobre SQL Server y PostgreSQL.

Se usaría especialmente si el negocio requiere menor latencia o procesamiento near real-time.

### Grafana

Grafana se utilizaría para visualización de métricas y dashboards de monitoreo.

Puede mostrar información como:

- Estado de pipelines.
- Duración de ejecuciones.
- Errores.
- Volumen de datos procesados.
- Métricas de Glue.
- Métricas de Airflow.
- Métricas de Kafka.
- Calidad de datos.

Grafana no reemplaza a Airflow. Se complementan: Airflow ejecuta y coordina; Grafana observa y visualiza.

# Lambda y EventBridge

Lambda y EventBridge se usarían como componentes serverless complementarios.

EventBridge puede programar o disparar eventos. Lambda puede ejecutar validaciones ligeras o reaccionar a eventos de S3.

Sin embargo, el control principal del pipeline se mantendría en Airflow.

## 5. Conclusión

La arquitectura propuesta en AWS permite consolidar datos de un CRM propietario, SQL Server y PostgreSQL en un Data Lakehouse basado en Amazon S3.

AWS Glue permite transformar los datos por capas, desde Landing hasta Gold, generando datasets confiables para consumo operativo y ciencia de datos. Amazon Athena habilita consultas SQL directamente sobre los datos publicados en Gold Operacional, cumpliendo el objetivo principal del área operativa.

Para ciencia de datos, la arquitectura genera datasets preparados en Gold Data Science que pueden consumirse desde SageMaker, notebooks o Spark ML para aplicar clustering. Además, se pueden generar relaciones cliente-producto, cliente-transacción y producto-categoría para análisis en motores de grafos como Amazon Neptune o Neo4j.

Apache Airflow, se utiliza como orquestador principal del pipeline diario, coordinando extracciones, transformaciones, validaciones, publicación de datos, actualización de catálogo y registro de auditoría.

Como componentes complementarios, Lambda y EventBridge permiten automatizaciones ligeras y disparadores event-driven; CloudWatch y Grafana permiten monitoreo y observabilidad; Kafka/MSK con Debezium puede incorporarse como evolución para CDC y procesamiento near real-time.

Finalmente, la arquitectura considera seguridad end-to-end mediante red privada, IAM, Secrets Manager, cifrado, Lake Formation, monitoreo, auditoría, control de accesos, clasificación de datos sensibles, control de metadata, control de cambios de esquema y linaje de datos.